

초음파 센서 칼만 필터 적용 가이드

ultrasonic_node.py에 1차원 칼만 필터를 추가하여 거리 측정값 안정화하기

개요

새 파일을 다시 만들 필요 없이 현재 `ultrasonic_node.py` 를 수정하면 됩니다. 가장 안전한 구조는 다음과 같습니다.



다만 생기부에서 전후 비교까지 하려면 원시값도 확인할 수 있어야 하므로, 다음 두 토픽을 발행하는 방식이 좋습니다.

토픽	내용
<code>/ultrasonic_distance_raw</code>	원시 거리
<code>/ultrasonic_distance</code>	칼만 필터 거리

기존 `auto_drive.py` 는 `/ultrasonic_distance` 만 구독하므로 수정하지 않아도 필터값을 사용하게 됩니다.

1. 먼저 백업

실행 중인 노드는 `Ctrl+C` 로 종료한 후:

```
cd ~/yahboomcar_ros2_ws/yahboomcar_ws/src/auto_drive/auto_drive
cp ultrasonic_node.py ultrasonic_node.py.bak_before_kalman
```

백업 파일은 `.py` 로 끝나지 않으므로 패키지 빌드에 영향을 주지 않습니다.

2. ultrasonic_node.py 열기

```
nano ultrasonic_node.py
```

3. 1차원 칼만 필터 클래스 추가

`UltrasonicNode` 클래스보다 위쪽에 다음 클래스를 넣습니다.

```

class KalmanFilter1D:
    """초음파 거리값을 위한 1차원 칼만 필터"""

    def __init__(
        self,
        process_variance=0.1,
        measurement_variance=4.0,
        initial_estimate=None
    ):
        # Q: 실제 거리 변화에 대한 불확실성
        self.process_variance = float(process_variance)

        # R: 센서 측정값의 불확실성
        self.measurement_variance = float(measurement_variance)

        # 초기 추정 오차 공분산
        self.estimate_error = 1.0

        # 초기 거리 추정값
        self.estimate = initial_estimate

    def update(self, measurement):
        measurement = float(measurement)

        # 첫 측정값을 초기 추정값으로 사용
        if self.estimate is None:
            self.estimate = measurement
            return self.estimate

        # 1. 예측 단계
        predicted_estimate = self.estimate
        predicted_error = (
            self.estimate_error + self.process_variance
        )

        # 2. 칼만 이득 계산
        kalman_gain = predicted_error / (
            predicted_error + self.measurement_variance
        )

        # 3. 측정값을 반영하여 추정값 보정
        self.estimate = predicted_estimate + kalman_gain * (
            measurement - predicted_estimate
        )

        # 4. 추정 오차 갱신
        self.estimate_error = (
            1.0 - kalman_gain
        ) * predicted_error

        return self.estimate

```

구조는 다음처럼 됩니다.

```

import rclpy
...

class KalmanFilter1D:
    ...

class UltrasonicNode(Node):
    ...

```

4. UltrasonicNode.__init__() 수정

현재 `__init__()` 안에 기존 Publisher가 있을 것입니다. 예를 들면:

```

self.publisher_ = self.create_publisher(
    Float32,
    '/ultrasonic_distance',
    10
)

```

그 부분은 유지하고, 원시값 Publisher와 칼만 필터 객체를 추가합니다.

```

# 필터링된 거리값: auto_drive.py가 사용하는 토픽
self.publisher_ = self.create_publisher(
    Float32,
    '/ultrasonic_distance',
    10
)

# 실험 비교용 원시 거리값 토픽
self.raw_publisher_ = self.create_publisher(
    Float32,
    '/ultrasonic_distance_raw',
    10
)

# 1차원 칼만 필터
self.kalman_filter = KalmanFilter1D(
    process_variance=0.1,
    measurement_variance=4.0
)

```

이미 `self.publisher_` 가 있다면 다시 만들지 말고, 그 아래에 `raw_publisher_` 와 `kalman_filter` 두 부분만 추가하면 됩니다.

5. 거리 측정 후 Publish 부분 수정

현재 코드에는 대략 다음과 같은 부분이 있을 것입니다.

```

distance = self.measure_distance()

if distance is not None:
    msg = Float32()
    msg.data = float(distance)
    self.publisher_.publish(msg)

    self.get_logger().info(
        f'Distance: {distance:.1f} cm'
    )

```

이 부분을 다음처럼 수정합니다.

```

raw_distance = self.measure_distance()

if raw_distance is None:
    return

# 사용할 수 없는 값 제외
if raw_distance <= 0.0 or raw_distance > 400.0:
    return

# 원시 거리 발행
raw_msg = Float32()
raw_msg.data = float(raw_distance)
self.raw_publisher_.publish(raw_msg)

# 칼만 필터 적용
filtered_distance = self.kalman_filter.update(raw_distance)

# 필터링된 거리 발행
filtered_msg = Float32()
filtered_msg.data = float(filtered_distance)
self.publisher_.publish(filtered_msg)

# 원시값과 필터값 동시 출력
self.get_logger().info(
    f'Raw: {raw_distance:.1f} cm | '
    f'Kalman: {filtered_distance:.1f} cm'
)

```

참고: 함수 이름이 `measure_distance()` 가 아닐 수 있습니다. 중요한 것은 기존 코드에서 거리가 계산되고 `/ultrasonic_distance` 로 발행되는 부분을 위 구조로 바꾸는 것입니다.

수정 후 핵심 코드 흐름

```

raw_distance = self.measure_distance()

if raw_distance is None:
    return

if raw_distance <= 0.0 or raw_distance > 400.0:
    return

raw_msg = Float32()
raw_msg.data = float(raw_distance)
self.raw_publisher_.publish(raw_msg)

filtered_distance = self.kalman_filter.update(raw_distance)

filtered_msg = Float32()
filtered_msg.data = float(filtered_distance)
self.publisher_.publish(filtered_msg)

```

이제 `auto_drive.py` 는 기존과 똑같이 `/ultrasonic_distance` 토픽을 받습니다. 하지만 이 토픽의 데이터는 기존 원시값이 아니라 칼만 필터가 적용된 거리값입니다.

6. 저장 및 검사

nano 저장:

```

Ctrl+O
Enter
Ctrl+X

```

문법과 들여쓰기 검사:

```
cd ~/yahboomcar_ros2_ws/yahboomcar_ws/src/auto_drive/auto_drive

python3 -m tabnanny ultrasonic_node.py
python3 -m py_compile ultrasonic_node.py
```

아무것도 출력되지 않으면 정상입니다.

7. 다시 빌드

```
cd ~/yahboomcar_ros2_ws/yahboomcar_ws

rm -rf build/auto_drive
rm -rf install/auto_drive

colcon build --packages-select auto_drive
```

정상적으로 끝나면:

```
source /opt/ros/humble/setup.bash
source install/setup.bash
```

8. 초음파 노드 실행

```
ros2 run auto_drive ultrasonic_node
```

다음처럼 출력되면 됩니다.

```
Raw: 16.6 cm | Kalman: 16.6 cm
Raw: 16.4 cm | Kalman: 16.6 cm
Raw: 26.1 cm | Kalman: 17.9 cm
Raw: 16.5 cm | Kalman: 17.6 cm
```

9. 두 토픽 확인

다른 터미널에서:

```
cd ~/yahboomcar_ros2_ws/yahboomcar_ws
source /opt/ros/humble/setup.bash
source install/setup.bash
```

원시값:

```
ros2 topic echo /ultrasonic_distance_raw
```

필터값:

```
ros2 topic echo /ultrasonic_distance
```

토픽 목록도 확인할 수 있습니다.

```
ros2 topic list | grep ultrasonic
```

정상 결과:

```
/ultrasonic_distance
/ultrasonic_distance_raw
```

10. auto_drive.py는 어떻게 되는가

기존 구독 코드가 다음이라면:

```
self.ultrasonic_subscriber = self.create_subscription(
    Float32,
    '/ultrasonic_distance',
    self.ultrasonic_callback,
    10
)
```

변경하지 않습니다. 자동으로 필터값을 받아 정지 판단에 사용합니다.



중요한 주의점

안전 주의: 칼만 필터는 순간적으로 잘못 커진 값은 완화하지만, 실제 장애물이 갑자기 가까워질 때도 값이 천천히 내려갈 수 있습니다. 그래서 첫 적용에서는 차량을 바닥에 바로 놓지 말고 **바퀴를 띄운 상태에서** 테스트해야 합니다.

파라미터 튜닝

증상	조정 방법
필터 후 장애물 감지가 늦음	<code>process_variance=0.1</code> → 0.5 로 높임
값이 여전히 많이 흔들림	<code>measurement_variance=4.0</code> → 9.0 으로 높임

처음에는 `process_variance=0.1` , `measurement_variance=4.0` 으로 시작하는 것이 무난합니다.